

DOINGG@UNION.EDU

DOING-LAB-NOTEBOOK

SELF

Contents

Home 7

Summer Research 2026! 7

Calendar 9

Calendar 9

Meeting Notes 11

Lab Meeting 6/15/26 13

Lab Meeting: Week 42 15

Lab Meeting 6/24/26 17

Lab Meeting: Week 43 19

Lab Meeting 6/29/26 21

Lab Meeting: Week 44 23

Weekly Notes 25

GD Week 42 27

GD Weekly Notes: Week 1 of Summer 26 29

Shared/Overall To-do 29

Phylo-learning To-do 29

Bio-xAI To-do 29

GD Week 43 31

GD Weekly Notes: Week 2 of Summer 26 33

Focus/Reading 33

<i>Still in bookkeeping mode / To-do / not yet to research notes ..</i>	33
<i>Shared/Overall To-do</i>	33
<i>Functional-learning To-do</i>	33
<i>Phylo-learning To-Do</i>	34
<i>Bio-xAI To-do</i>	34
 <i>Working Notebook</i>	 35
<i>Week 1</i>	35
 <i>Article Notes: “ADAGE signature analysis: differential expression analysis with data-defined gene sets” (Tan et al. 2017)</i>	 37
<i>Pseudomonas aeruginosa</i>	37
<i>ADAGE signature analysis pipeline</i>	38
<i>Methods</i>	38
 <i>Proposal Notes</i>	 41
<i>6/25/2026</i>	41
<i>Main points</i>	41
<i>To Do List</i>	42
<i>DNA Seq Filtering</i>	42

GD Week 44 43

Processing Tn-seq Data 45

Previously 45

This Week 46

Assemble new genomes, collect public genomes 47

Prepare genomes 51

Prepare plamids 55

Prepare Plasmid 57

Process reads 59

Next up, process reads 61

Misc Notes 63

6 Jupyter Notebook Habits That Make Your Work Easy to Return To 65

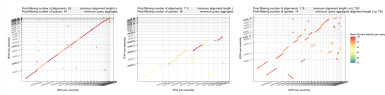
6 Jupyter Notebook Habits That Make Your Work Easy to Return To 67

MT proposal 77

SP proposal 83

ES proposal 89

Home



The shared electronic lab notebook of the Doing Comp-Bio Lab.

Summer Research 2026!

Calendar

Calendar

(read only, edit in [Google Calendar](#))

Meeting Notes

Lab Meeting 6/15/26

Lab Meeting: Week 42

June 15, 2026

1. lab meeting structure (first few weeks)
 - 10-15 mins each for project Q&A
 - maybe walk through notebooks from the previous week, if helpful
 - flash journal club
2. daily scrum - slack
 - home or lab
 - what you'll focus on
 - to know what Qs to expect or work together
 - to avoid doing the same thing
 - to get ideas from each other
 - approx hours
 - to synchronize if helpful
 - to know when to expect Qs or As
3. working notebooks vs. notebook notebooks
 - Medium post on [good jupyter nb habits](#)
 - working colab for now - for reproducibility
 - data stored in the same google drive folder should be reproducible across accounts..
 - Union supposed to be getting a compute cluster
 - notebook notebooks can be edited in anything
 - I use VS code with googlde drive desktop synching files

- could also use colab notebooks, just different ones from working/explorations notebooks
 - could also use online editor that allows md format (eg. Text Editor google drive app)
 - don't require reading in data?
4. github seqADAGE <https://github.com/georgiadoing/seqADAGE>
 - fork
 - clone
 - make project folder within /Py/
 - we'll start code review in a couple weeks
 - [github tutorials](#)
 - maybe try the first..
 5. data in google drive folder (for now)
 - later will use Open Science Framework: <https://osf.io/>
 - maybe [this one](#) but maybe a *new* one
 - could create accounts (union email?)
 6. Misc.
 - my office key and keycard

Lab Meeting 6/24/26

Lab Meeting: Week 43

June 24, 2026

1. Lab Announcements

- schedule 1-1s (tomorrow? next mon/tues?)
- Seminar tomorrow 11:30, Olin 115: **Kathryn Feller - Seeing and Stabbing: A mantis shrimp journey**
- also student social committee, other announcements from Matt/Undergrad Research Office?
- next week's sched
 - lab meeting Monday again
 - I'll be around Monday, Tuesday and remote Wed, Thursday
 - Friday off as "4th" of July

2. Last week's review

- how did seminars/events go?
- did "scrum" work/help?
- logistics working out? (keycards, housing, food, driving/parking)
- workday hours?

3. Project updates

- note pages to add to notebook site?
- function t-learning (Mukundan)
- shap vals (Seo)
- questions? hurtles? sucessess? next steps?

4. Flash JC

- overview: 5 min total
 - Title, journal, publish date, author
 - Most compelling finding (1 or 2 sentence(s))
 - Figure (panel) that convinces you *the most*
 - Background needed to interpret *just that figure*
 - Outstanding questions/concerns/implications
- Emma: [ADAGE signature analysis, BMC Bioinformatics 2017](#)
- Mukundan: [AlphaFold, Nature 2021](#)
- Seo: [Rabbit evolution, Gene 2024](#)

Lab Meeting 6/29/26

Lab Meeting: Week 44

June 29, 2026

1. Lab Announcements

- Seminar tomorrow 11:30, Olin 115: **Project “Speed Dating”**
 - think ahead to flash-talks...
 - free lunch!
- 'll be around Monday, Tuesday and remote Wed, Thursday
- Friday off as “4th” of July
- schedule 1-1s (tomorrow? next mon/tues?)

2. Project updates

- note pages to add to notebook site?
- function t-learning (Mukundan)
- shap vals (Seo)
- phylo t-learning (Emma)
- questions? hurtles? sucessess? next steps?

3. Flash JC

- slides or notebooks (optional)?
- overview: 5 min total
 - Title, jounral, publish date, author
 - Most compelling finding (1 or 2 sentence(s))
 - Figure (panel) that convinces you *the most*
 - Background needed to interpret *just that figure*
 - Outstanding questions/concerns/implications

- Emma: [ADAGE signature analysis](#), BMC Bioinformtics 2017
- Mukundan: [AlphaFold](#), Nature 2021
- Seo: [Rabbit evolution](#), Gene 2024

Weekly Notes

GD Week 42

GD Weekly Notes: Week 1 of Summer 26

June 15, 2026

Shared/Overall To-do

- ☐ Implement tuner method in adage class
- ☐ E coli gene lists for enrichment analyses
- ☒ Get lab notebook setup
- ☐ Decide on repo structure
- ☐ Plan sampling excursion
- ☒ Key access to CAP/crochet
- ☒ Schedule lab meetings - update calendar
- ☒ Group meeting once a week
- ☐ 1-1 meetings also weekly or as-needed

Phylo-learning To-do

- ☒ Get DNA sequences into google drive
- ☐ Get P.a. DNA seq
- ☐ S aureus gene lists for enrichment analysis
- ☐ P aeruginosa gene lists for enrichment analysis

Bio-xAI To-do

- ☐ model to test SHAP tutorial on

☐ colaboration with Josh/Ara

GD Week 43

GD Weekly Notes: Week 2 of Summer 26

June 24, 2026

Focus/Reading

- Joan Didion, *On keeping a notebook*, *Why I write*
- connect to scientific notebooks...?... still working in it

Still in bookkeeping mode / To-do / not yet to research notes ..

- ☐ Connect with Julia Oh (Duke) over Tn-seq project/data collab
- ☐ Connect with Josh Chuah (BME) over SHAP collab
- ☐ Connect with Emily and Memo (NEB) over E coli collab
- ☐ Connect with Carol Weiss (Psychology) over data analytics collab

Shared/Overall To-do

- ☐ Implement tuner method in adage class
- ☐ E coli gene lists for enrichment analyses
- ☐ Decide on repo structure
- ☐ Plan sampling excursion

Functional-learning To-do

- ☐ Get P.a. DNA seq

- ☐ S aureus gene lists for enrichment analysis

Phylo-learning To-Do

- ☐ P aeruginosa gene lists for enrichment analysis

Bio-xAI To-do

- ☐ model to test SHAP tutorial on
- ☐ colabration with Josh/Ara

Working Notebook

Week 1

Article Notes: “ADAGE signature analysis: differential expression analysis with data-defined gene sets” (Tan et al. 2017)

- identify bio processes affected by Anr on CFBE cells

Pseudomonas aeruginosa

- wildtype and Δ anr mutant cells
 - Anr is a transcriptional regulator responsible for the aerobic to anaerobic transition
 - * Δ Anr mutants cannot switch from aerobic to anaerobic respiration (loss of anaerobic growth)
 - * can't form biofilms (usually low in oxygen)
- grown on cystic fibrosis genotype bronchial epithelial cells
 - models cystic fibrosis airway infections
- has sufficient public gene expression data to construct model
- rapidly growing pathway annotations allow them to validate bio relevance of gene signatures learned by ADAGE
 - also allows demonstration of ADAGE's ability to identify unannotated bio processes

ADAGE signature analysis pipeline

- ID one or more signatures that respond to experimental treatment (signatures represent bio processes that may be affected by treatment)
 - similar to traditional gene set analysis but replaces human-annotated gene sets with ADAGE-learned signatures
- developed R package and web server

Methods

Data preparation (Fig 1) * Raw microarray data analyzed with RMA (Robust Multi-array Average) * background correction (removes noise), quantile normalization (align distributions to compendium), probe summarization (ask Georgia???) * assigned expression of missing genes using k-nearest neighbor based on similarity to compendium * RNA-seq expression values normalized to compendium via TDM (Test DATA Management) * generate synthetic data or scramble real data to protect privacy while preserving statistical properties needed for model training * track changes in datasets so developers can reproduce results * pull manageable, representative portions of large datasets for faster, cheaper model testing * zero-one normalization * compendium as reference * measurements outside range observed in reference are set to 0 or 1 according to whether they're below or above range

Active signature detection

- signature's activity reflects how active that signature is in each sample
 - average expression values of signature genes weighted according to their weight in the signature
 - Heat map in Fig 1

Signature interpretation

- only need to examine signatures affected by experiment, not ALL signatures
- signatures are NOT annotated to specific bio process
 - can infer bio processes by looking at gene composition if some gene have been previously characterized and share bio theme
 - can also link existing knowledge (ie KEGG pathways and GO terms) to signatures through enrichment test

- * GO terms: standardized, controlled vocabulary phrases to describe gene functions and products (proteins or non-coding RNAs) across all species

Results (Fig 4)

- example dataset (wildtype vs Δ Anr cells)
- chart a
 - volcano plot of activity differences and significance
 - red-highlighted signature lie on top 10 layers of Pareto fronts
 - * pareto fronts: optimal solutions
 - * top layer (layer 1) = best solution (improving one objective inherently worsens another)
 - * other layers are suboptimal solutions (improved upon by previous layers)
 - red dotted line indicates 0.05 significance cutoff
- chart b
 - heatmap of 36 signatures (top 10 pareto layers) and their activities in each sample
 - * yellow = high activity, blue = low activity
 - signatures have more overlapping genes when model is trained with more noise
- chart c
 - heatmap color depicts how strongly the signature in the row is differentially activated when the genes it shares with the signature in the column have been depleted
 - * diagonal shows activation significance of a signature without overlapping gene removal (see chart d)
 - X = non-significant p-values

Fig. 5

- 19 differentially active signatures
- Sets of genes that have transcriptional relationships
 - Clusters can reveal regulons (multiple genes across chromosome that are regulated as a unit by the same transcription factor)
- Linked signatures to KEGG pathways (suggests the biological basis of signatures)

- 12/19 signatures were significantly enriched for one of more of 14 KEGG pathways
- Tested whether these KEGG pathways were differentially active between wild-type and delta-anr mutants
 - Only used shared genes and their associated pathways
 - Genes in 7/14 pathways were significantly activated
- Also performed GSEA
 - 5 pathways were detected by GSEA and ADAGE
 - * Have been shown to be regulated by Anr

Proposal Notes

6/25/2026

Main points

- QUESTION: Does strain-level transfer learning yield results that are more concordant with biological databases, such as EcoCyc or pseudomonas.com, than species-level and genus-level transfer learning?
 - Georgia's done some species-level research
- E.coli and P.aeruginosa
- can't use gene expression data of entire species because of strain variation
- EPEC (enteropathogenic E.coli) and EHEC (enterohemorrhagic E.coli)
 - increased virulence, antibiotic-resistance, significant causes of mortality
- if findings support hypothesis, they may suggest we could bridge the gap between lab-studied and disease-causing strains of E.coli
 - future studies could explore extent to which ML can be transferred (ie lab mice to humans) # Plan

1. data setup / filtering
2. transfer > 2.1 pre-training > 2.2 fine tuning > 2.3 assessment
3. output analysis > 3.1 look for enrichment between output and known pathways on pseudomonas.com and KEGG pathways » 3.1.1 gene-

pathway mapping used to evaluate biological relevance of gene signatures > 3.2 compare outputs across different pairs of pre-training and fine-tuning datasets to determine workflow

To Do List

- make sure we have all data needed
 - is E.coli strain data in the shared drive?
 - * mentioned EPEC and EHEC strains in proposal – do we have data for these?
 - do we have the P.aeruginosa data?
- figure out filtering
 - read papers on DNA seq filtering?
 - go through data and see what labels are there, then decide
 - * do all files in data folder have the same labels? If not, keep shared labels and delete any that aren't shared?
 - filter all data before starting with ML model
- figure out code – what changes do I need to make?
 - look through it and make sure you understand what's happening / how it works
 - brainstorm what you need to change / add
- explore pseudomonas.com and KEGG pathways
 - read papers and stuff about this?
 - get a feel for it before jumping right in with experiment output
 - maybe start with the Tan et al. (2017) paper and try to follow what they did?

DNA Seq Filtering

- What is in data folder?
 - which file am I using to start (and then filter from that file)?
 - what are rows and columns in the files?
- [Trimming and filtering data \(LinkedIn\)](#)
 - short reads - reads below a certain length threshold
 - duplicate reads - redundant sequences
 - high-N content reads - reads with too many ambiguous bases("N")
 - there's bioinformatics tools to help with filtering
 - * can we use them or are we filtering manually?

GD Week 44

Processing Tn-seq Data

I am continuing to process the third round of Tn-seq data for the Oh lab. This is the first run since Matt left so I am using his notes but have more steps to implement than previously. This is also a larger run with more strains and using a different plasmid, so we'll hit all the steps.

The pipeline follows as:

1. prepare genomes
2. prepare plasmids
3. prepare reads
4. run the numbered scripts in order

Previously

- using [Matt's benchling notebook(<https://benchling.com/s/etr-OjVpptIumXPuk6QrTx0S/edit?m=slm-lvLKsJ8OYNZeEpnM6j1C>)] as a reference
- I had previously "prepared reads" but converting from bcl to fastq
 - used bc12fastq module on DCC
 - copied from script in Matt's folder
 - used Illumina adapters: <https://support-docs.illumina.com/SHARE/AdapterSequences/Content/Illumina-UDIndexes.htm>
 - sample sheet from Aaron, reformatted: founders2_sample_sheet3.csv
 - shared read counts Sequencing Map 4.14.26_reads_edit.xls on slack

- I had the bash scripts working from previous experiments

This Week

- prepare the genomes and plasmids
- process the reads
- save data and reference sequences in permanent/accessible place

data so far

first run: ? FOUNDERS: /datacommons/ohlab/data_mWGS/Elwell_11300_251113B9/

next founders: /hpc/group/ohlab/yk279/Founders1.zip (260422_MN00462_0188_A00HCHGVM
)

Assemble new genomes, collect public genomes

- some were recently sequenced
- cp new_genomes from data_commons

Strain ID (oh lab)	Strain Name (external)	RefSeq prefix	Oh lab Seq	Used public	Used internal	From original paper
MG1655	MG1655	U0096				T
MDS42	MDS42	AP012306				T
RN4220	RN4220	GCF_018722030.1				T
SEPI19	NIHLM095	GCF_008276445.1				T
SEPI20	NIHLM127	SEPI20				T
SEPIA12228	ATCC12228					T
SEPI20	Tu3298					T
SEPI02	NIHLM003	GCF_008276445.1				T
SEPI03	NIHLM008	GCF_008276445.1				T
SEPI04	NIHLM015	GCF_008276445.1				T
SEPI06						T

Strain		RefSeq prefix	Oh lab Seq	Used pub- lic	Used inter- nal	From original paper
Strain ID (oh lab)	Strain Name (exter- nal)					
SEPI13	NIHLM 053				T	

- process with trimmomatic, spades, prokka

```
conda activate spades

export PATH=$PATH:/home/doingg/miniconda3/envs/spades/bin

for i in Sepi_19* ; do
file1="${i}" ;
file="${file1%%_R1_001.*}" ;
echo "${file}" ;
singularity exec docker://staphb/trimmomatic trimmomatic PE \
-trimlog /work/gd134/new_genomes/"$file".trim.log \
/work/gd134/new_genomes/"$file"_R1_001.fastq.gz \
/work/gd134/new_genomes/"$file"_R2_001.fastq.gz \
/work/gd134/new_genomes/"$file"_trimmedFp.fastq \
/work/gd134/new_genomes/"$file"_trimmedFup.fastq \
/work/gd134/new_genomes/"$file"_trimmedRp.fastq \
/work/gd134/new_genomes/"$file"_trimmedRup.fastq \
ILLUMINACLIP:TrueSeq3-PE.fa:2:30:10:2:True LEADING:3 TRAILING:3 MINLEN:36 2> /work/gd134/new_genomes/"$file".trim.log
spades.py -1 "$file"_trimmedFp.fastq -2 "$file"_trimmedRp.fastq --isolate -o "$file"_out
done
```

```
conda activate prokka

for i in Sepi_[A-Z1-9c]*_out ; do
file1="${i}" ;
file="${file1%%_gDNA_S[0-9][0-9]_out}" ;
echo "${file}" ;

prokka --outdir "${file1}/${file}_prokka" --kingdom 'Bacteria' --addgenes --locustag "$file" --prefix "$file"
done
```

- move nt fasta from /work to /hpc/group and rename
 - SEPI13

- SEPI06
- SEPIA12228
- SEPITU

- e.g.

```
# in /hpc/group/ohlab/doingg/tn-seq_data/NGS_ref_genomes
mkdir SEPI13
cp /work/gd134/new_genomes/Sepi_13_gDNA_S27_out/contigs.fasta SEPI13/SEPI13.fna
cp -r /work/gd134/new_genomes/Sepi_13_gDNA_S27_out/Sepi_13_prokka SEPI13/
```

- others are public, download, move rename
 - SEPI02
 - SEPI03
 - SEPI04
- from '/hpc/group/ohlab/doingg/tn-seq_data/NGS_ref_genomes'
- e.g.

```
conda activate roary

datasets download genome accession GCF_000276165.1 --include gff3,rna,cds,protein,genome,seq-report
unzip ncbi_dataset.zip
# [N] to not overwrite files

mv ncbi_dataset/data/GCF_* .
mkdir SEPI02
cp -rv GCF_000276165.1/* SEPI02
```


Prepare genomes

- from [Matt's benchlin notes](#), have to go from fastq and gff to bed files and bowtie2 indices

1. get fasta and gff files

```
conda activate ngs_processing_source_fastp

# from Matt
mv GCA_000014805.1_ASM1480v1_genomic.fna AP012306.fa
mv GCA_000014805.1_ASM1480v1_genomic.gff AP012306.gff3

mv AP012306.fna AP012306.fa
mv AP012306.gff AP012306.gff3

bedtools sort -i AP012306.gff3 > AP012306_sorted.gff3
```

- try with shorter name for public genomes, should match assembled OK
- e.g.

```
cp SEPI02/GCF_000276165.1_ASM27616v1_genomic.fna SEPI02/NZ_AKHB01000001v1.fna
cp SEPI02/genomic.gff SEPI02/NZ_AKHB01000001v1.gff3

cp SEPI02/GCF_000276165.1_ASM27616v1_genomic.fna SEPI02/SEPI02.fna
cp SEPI02/genomic.gff SEPI02/SEPI02.gff3
```

```
# for assembled, also move and rename
cp SEPI13/Sepi_13_prokka/Sepi_13.gff SEPI13/SEPI13.gff3
```

2. sort gff3

- need to delete data but out of gff3 for newly assembled/annotated genomes

```
conda activate ngs_processing_source_fastp

bedtools sort -i U00096.gff3 > U00096_sorted.gff3
bedtools sort -i AP012306.gff3 > AP012306_sorted.gff3
# 3. gff3 to bed

gff2bed < U00096_sorted.gff3 > U00096.bed
gff2bed < AP012306_sorted.gff3 > AP012306.bed

ls -lh AP012306.bed
head AP012306.bed

# 4. make genome files
samtools faidx U00096.fa
cut -f1,2 U00096.fa.fai > U00096_genome_file.txt

samtools faidx AP012306.fa
cut -f1,2 AP012306.fa.fai > AP012306_genome_file.txt

cat U00096.genome
# 5. bowtie2 indices

bowtie2-build U00096.fa U00096
bowtie2-build AP012306.fa AP012306
```

- putting it together for new genomes

```
conda activate activate ngs_processing_source_fastp
conda install -c conda-forge bedops

# save sortd as gff for dev to be able to rerun without dups
for i in SEPITU*/*.gff3 ; do
file1="${i}" ;
```

```

file="${file1%%.gff3}" ;
echo "${file}" ;
bedtools sort -i "${file1}" > "${file}_sorted.gff" ;
gff2bed < "${file}_sorted.gff" > "${file}.bed" ;
samtools faidx "${file}.fna" ;
cut -f1,2 "${file}.fna.fai" > "${file}_genome_file.txt" ;
bowtie2-build "${file}.fna" "${file}"
echo "${file}"
done

# change gff to gff3
for i in SEPTU*/*_sorted.gff ; do
file1="${i}" ;
file="${file1%%.gff}" ;
echo "${file}" ;
cp "${file1}" "${file}3"
done

```

6. process to PGT

- jupyter notebook copied from /hpc/group/ohlab/doingg/tn-seq_data_analysis/src/accessory_scripts to /hpc/group/ohlab/doingg/tn-seq_ohlab/src/accessory_scripts
 - ref_genome_processingGD.ipynb
 - gff_to_bed2.py

Prepare plamids

plasmid	sequence	function	source	oh lab experi- ments	drug(s)
pENG1	pENG- 131_Helper_pTet- Himar1C9_p15A_Amp.gb	helper	gb in code from paper	founders	amp
pENG2	pENG- 132_Tn[himar1c9]_Autonomous_pTet_Mariner_Chlor.gb			first	chlor
pENG3	pENG- 133_Helper_pTet- Mariner_med_copy.gb	helper			
pENG4	pENG- 134_Helper_pBAD- Mariner_med_copy.gb	helper			
pENG5	pENG- 135_Tn_Donor_KanR_v2.gb	donor			kan
pENG6	pENG- 137_Tn[GFP]Donor_mNeonGreen_v2.gb	donor			
pENG146	pENG-146_Tn- pOUT_Donor_pJ23104.gb	donor		first staph	
pENG150	pENG- 150_Tn[RFP]_Donor_mCherry_CamR_v2.gb	donor			cam
pENG152	pENG- 152_Tn[sfGFP]_Donor_sfGFP_KanR_v2.gb	donor		first staph, founders	kan

plasmid	sequence	function	source	oh lab experi- ments	drug(s)
pMPTnV2	Donor_RB- TnV2 (same as pMP_TnV2_barcode.gb) pMP_RB- TnV2_plac01- pL_BBa_R0011_barcode.gb pMP_RB- TnV2_pLEx_EliR_PDE20_barcode.gb		addgene 422622	founders3	

Prepare Plasmid

- based on the plasmid sequence from addgene (<https://www.addgene.org/browse/sequence/422622/>), the plasmid used in this latest experiment seems like an RB plasmid
 - is this helper or donor?
 - used blast, clustal, muscle
 - got fasta for alignment from gb for the RB plasmids with snapgene free trial, although Matt did have code to do so for other plasmids

```
conda create -n gbconvert -c conda-forge biopython bcbio-gff
conda activate gbconvert
```

- instead i'll just add these libs to ngs_processing env
- from /hpc/group/ohlab/doingg/tn-seq_data/NGS_ref/genomes

```
conda activate ngs_processing_source_fastp
conda install -c conda-forge biopython bcbio-gff
# rename plasmid to shorter name
cp plasmid_maps/pMP_TnV2_barcoded.gb pMPTnV2.gb
```

```
# from Matt
python -c "from Bio import SeqIO; SeqIO.convert('/hpc/group/ohlab/doingg/tn-seq_data/NGS_ref_genomes/plasmid_maps/pMP_TnV2_barcoded.gb', 'genbank', 'fasta', 'pMPTnV2.fasta')"
python -c "from Bio import SeqIO; from BCBio import GFF; rec=SeqIO.read('pMPTnV2.gb', 'genbank'); GFF.write(rec, 'pMPTnV2.gff', 'gff')"
bowtie2-build pMPTnV2.fasta pMPTnV2
```


Process reads

- old results did not save (on /work), need to re-process them
- from within tn-seq_ohlab/src/
- outputs written to /work/gd134/tn-seq_data/tn-seq_outputs
- need to expand archives and make output dir

```
gunzip /work/gd134/tn-seq_data/*.fastq.gz
mkdir /work/gd134/tn-seq_data/tn-seq_outputs/staph
```

```
conda activate tnseq
```

```
./01_fastp_align_fastq_GD.sh ../metadata_staph.txt /work/gd134/tn-seq_data/*R1_001.fastq
```

```
# now to try new experiment
```

```
./01_fastp_align_fastq_GD.sh ../metadata_founders_exp3_test.txt /work/gd134/fastq_output2/*R1_001.fastq
```

*** will need to manually add read counts to metadata file under “total_reads_postQC”

Next up, process reads

e.g.

```
cd accessory_scripts
# 1
./deduplicate_bam_files_GD.sh ../../metadata_staph.txt /work/gd134/tn-seq_data/tn-seq_outputs/staph/bow
# 2
./calculate_percent_mapped_reads.sh /work/gd134/tn-seq_data/tn-seq_outputs/staph/fastp_output/*_out_R1.
# 3
python3 deduplication_stats.py -i /work/gd134/tn-seq_data/tn-seq_outputs/staph/bowtie2_output_dedup
cd ..
# 4
./02_pygenometricks_make_tracks_GD.sh ../../metadata_staph.txt /work/gd134/tn-seq_data/*R1_001.fastq
# 5
python3 03_pygenometricks_edit_tracks.py -i /work/gd134/tn-seq_data/tn-seq_outputs/staph/pygenometricks
# 6
./04_pygenometricks_viz_tracks_GD.sh ../../metadata_staph.txt /work/gd134/tn-seq_data/*R1_001.fastq
# 7
python3 05_peak_annotation_GD.py -i /work/gd134/tn-seq_data/tn-seq_outputs/staph/macs3_output/
# 8
python3 05_peak_annotation_GD.py -i /work/gd134/tn-seq_data/tn-seq_outputs/staph/macs3_output_dedup/
```


Misc Notes

6 Jupyter Notebook Habits That Make Your Work Easy to Return To

From messy exploration to a record that still makes sense months later

6 Jupyter Notebook Habits That Make Your Work Easy to Return To

From messy exploration to a record that still makes sense months later

Published Mar 07, 2026 · Updated Apr 20, 2026 · Free: No

- **PDF version**

Here is a situation working in Jupyter Notebooks that many people will recognise.

A colleague messages you asking how a particular calculation was carried out in a study that was worked on six months ago. You know the answer is somewhere in your jupyter notebook.

You open it, confident the logic and information is in there.

Forty minutes later, you are still not sure.

The code ran. The outputs exist. But the reasoning behind the key decisions is gone. Which Clay Volume cutoff did you use and why? Was normalisation applied to the gamma ray data before or after the formation tops were picked? There is a commented-out cell that might be relevant, or might be an abandoned experiment. You can't tell.

Then you start to notice that a cell further down the page has errored as

a variable is not present. After much head scratching you find out it was calculated 5 cells further down.

This is not a messy notebook problem. Messy exploration is normal and expected. This is a documentation problem: the analysis produced a result, but it did not produce a record.

In subsurface work, that distinction carries real weight. Notebooks often sit at the base of interpretations that can feed future decisions, reserve estimates, and machine learning model training pipelines. When someone asks why a decision was made, “I think that was the approach” is not an audit trail. It is a gap.

These six habits don’t slow down exploration. They ensure that what comes out of it is traceable, rerunnable, and readable by someone who was not in the room when it was written, including you, six months later.

This article does have a geoscience twist to it, but these habits can be applied to any data science project.

1. Treat the Notebook as a Decision Log, Not a Script

This is the shift that changes everything else.

Most notebooks are written as scripts: a sequence of code cells that produces an output. The problem with a script framing is that it records *what happened*, but not *why*. Code tells you that Gamma Ray values below zero were removed. It doesn’t tell you whether that was a deliberate interpretation choice, a quality control step, or a temporary workaround that was never revisited.

A notebook written as a decision log treats every significant step as something that needs to be justified, not just executed.

The practical difference is in how you use markdown cells. Not as labels for the code below them, but as the reasoning that precedes it.

Compare these two approaches to the same step.

Label (not enough):

```
## GR Cleaning
Removing negative values and normalising.
```

Decision log (what you actually need):

```
## GR Curve: Cleaning Decisions
Negative GR values present in 3 wells (n=14 samples total). Spike pattern
and depth distribution consistent with tool noise rather than formation signal.
Excluded prior to any formation-level analysis.

Normalisation: min-max applied rather than z-score. Reason: preserves the
shape of the GR distribution across formations, which matters for the
cross-plot in section 4. Z-score would compress the tails where the
lithological contrast is most visible.
```

The code that follows executes the decision. The markdown records it.

When you return to this notebook in four months, you don't need to reverse-engineer the reasoning from the code. It is written down. When a colleague picks it up, they know not just what was done but why, and under what assumptions.

A useful test: if you cannot write a defensible markdown cell for a step, that is a signal the decision itself needs more thought before you commit it to code.

2. Write the Header Before You Write Any Code

At the start of any notebook, you can save yourself time later on by spending two minutes writing a header cell before writing a single line of code.

```
# Project: Volve Field -- Petrophysical QC
## Notebook: 03_formation_analysis -- Curve Completeness Assessment

**Purpose:**
Assess GR and RHOB curve completeness by formation across the Volve dataset.
Output used to determine viable curve inputs for the lithology classification model.

**Input:**
`data/raw/volve_well_logs.csv`
Source: Volve open dataset, DISKOS export, retrieved 2026-01-15.
Wells in scope: 15/9-F-1, 15/9-F-4, 15/9-F-11 (producers only).

**Assumptions and exclusions:**
Intervals above casing shoe depth excluded (varies by well, see formation tops file).
GR values < 0 treated as noise and removed prior to analysis.
Brent Group subdivisions collapsed to group level due to sparse tops coverage.
```

```

**Output:**
`data/processed/curve_completeness_by_formation.csv`
`outputs/figures/completeness_heatmap.png`

**Last updated:** 2026-02-26 | **Status:** Complete

```

The two sections that matter most in geoscience or data science work are data provenance and assumptions.

Data provenance answers: where did this data come from, when was it obtained, and exactly which subset are you working with. A file path tells you nothing about whether the data came from a live database pull last week or a spreadsheet that was emailed across six months ago. Source and scope belong in the header because they are invisible everywhere else.

Additionally, a file path that resolves on your machine and nowhere else is not provenance. It is a dead end. If the data lives anywhere other than a shared location the next person is guaranteed to have access to, the header needs to explain how to get it:

```

**Input:**
`data/raw/volve_well_logs.csv`
Source: Volve open dataset, DISKOS export, retrieved 2026-01-15.

Wells in scope: 15/9-F-1, 15/9-F-4, 15/9-F-11 (producers only).

Access: Shared project drive at \\server\projects\volve_qc\data\raw\
If unavailable, contact data management or re-export from DISKOS using
the query parameters documented in `docs/data_export_notes.md`.

```

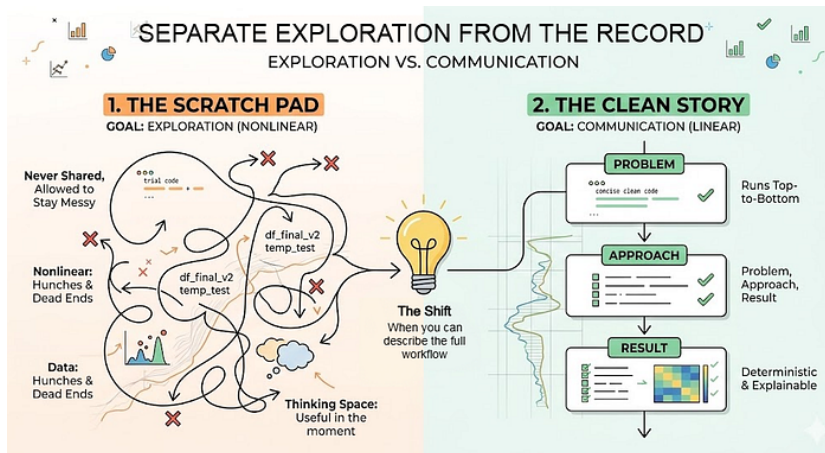
The fields that matter when access is not guaranteed: where the file physically lives or can be retrieved from, who to contact if access is unavailable, how it was originally obtained so it can be re-obtained the same way, and any query parameters or export settings used. A re-export with different settings is not the same dataset, and that distinction will not be obvious to the person picking up the notebook six months later.

Assumptions and exclusions are where the interpretive choices live. Collapsing Brent Group subdivisions is a meaningful geological decision. Excluding above-casing intervals is a meaningful QC decision. Both are defensible. Neither shows up in the code without a comment, and comments are the first thing to disappear during a cleanup. The header cell is

permanent.

A simple status message can inform you and other notebook users whether this is a complete notebook or if it is still a work in progress.

3. *Separate Exploration From the Record*



Exploration and documentation are not the same activity, and trying to do both in one notebook produces something that serves neither purpose well.

Exploration is nonlinear by nature. You are testing approaches, hitting dead ends, trying variations you will ultimately discard. That process should be unconstrained. A notebook full of experimental cells, abandoned branches, and intermediate print statements is not a problem. It's how analysis actually happens.

The problem comes when that exploratory notebook is also treated as the record. It looks like an analysis. It produces outputs. But the path through it is not a story. It is a session log.

The habit is to maintain two notebooks per project.

The working notebook is where exploration lives. It has no cleanliness requirement. Dead ends stay in. Commented-out experiments stay in. Variable names that only made sense at 2am stay in. It is never shared and never needs to be.

The record notebook is written once you understand the shape of the answer. It tells one story: problem, approach, result. It runs cleanly from top to bottom. Every significant decision has a markdown cell. Outputs are

saved to predictable locations.

A practical signal for when to make the transition: when you can describe the approach out loud, in sequence, without referring to the notebook. If you can't do that yet, you are still likely in exploration territory.

The record notebook is not just a cleaned-up version of the working notebook. It should be written from scratch, with the benefit of already knowing what the analysis is and what functions or code snippets matter from your exploration notebook. That distinction matters. Cleaning up an exploratory notebook tends to preserve its structure while hiding its confusion. Writing the record from scratch produces something that actually reads as intended and can also highlight any issues in your initial analysis.

The record notebook matters even more as AI starts to enter the analysis workflow. A model can help generate interpretations, but someone still has to verify them. And that verification depends on having a clear record of what decisions were made and why. I've written about why verification becomes the hardest problem in technical AI work over on Substack. Link at the end.

4. Name Variables for the Person Who Comes Back

Variable naming is documentation that can't be separated from the code. Unlike comments, it can't be deleted. Unlike markdown cells, it is present on every line that references the object.

The goal isn't descriptive names for their own sake. It's names that carry two pieces of information: what the object contains, and what stage of the workflow it represents.

A stage-prefix convention makes both visible at a glance:

```
raw_logs      # loaded directly from file, unmodified
clean_logs    # nulls removed, depth range trimmed, curves validated
logs_with_vcl # Vcl appended from a clay volume calculation
train_logs    # subset used for model fitting
test_logs     # held-out subset, not seen during training
```

For model objects, include the algorithm and the target variable:

```
model_rf_lithology    # random forest classifier, predicts lithology class
model_lgbm_permeability # LightGBM regressor, predicts permeability (mD)
```

The rubric is simple: a name should tell you what it is and where it sits in the workflow. If you can't construct a name that does both, the object itself may not be clearly defined yet. That ambiguity is worth resolving before writing code that depends on it.

One habit that avoids the creeping `df2`, `df3`, `df_final` problem: name the object at creation, not at use. If you plan to rename it later, you will very unlikely rename it later.

5. The Kernel Restart Is Not Optional

Before closing any record notebook, restart the kernel and run all cells from top to bottom. If it doesn't complete cleanly, it is not finished.

The most common notebook failure mode is hidden state: a variable that was defined in an earlier session and no longer exists in the code, a file loaded from a path that no longer resolves, an import that worked once because another library happened to already be in memory. In the current session, everything appears correct. On a different machine, or six months later, it fails silently or not so silently at a cell with no obvious explanation.

The restart-and-run check is the only reliable way to surface these problems before they surface for someone else.

The practical objection is compute time. Some steps are expensive and re-running a full model training pass or reloading a large dataset on every review pass isn't realistic. The pattern that handles this cleanly is to cache intermediate outputs to `data/processed/` and load from cache in subsequent steps. The record notebook reads the cached output rather than re-executing the expensive step. The reproducibility standard still holds: the notebook runs top-to-bottom, deterministically, it just does not re-derive everything from raw data in a single pass.

The distinction worth being clear on: a notebook that produces the same outputs given the same inputs is reproducible, regardless of whether it uses cached intermediates. A notebook that only works when run in a specific order, with specific variables already in memory from a previous session, is neither reproducible nor fast. It is a future debugging problem waiting to surface at the worst possible moment.

6. Apply the Cold Open Test Before Closing

The first five habits are about writing the notebook. This one is about checking whether it actually works without you.

Before treating any record notebook as complete, try this: close it, open it fresh, and ask whether a colleague could follow it without asking you a single question. Not a full peer review. Just a five-minute scan. Can they identify the purpose from the header? Can they locate the key decisions in the markdown? Do they know what to do with the outputs?

If the answer to any of those is no, the notebook is not finished regardless of whether it runs cleanly.

In subsurface work this test has particular weight because notebooks routinely outlive the context in which they were written. A petrophysicist moves to another asset, goes on leave, or leaves the organisation. Six months later someone needs to understand what was done and why, and the only record is the notebook. It has to stand on its own.

A practical checklist to run before marking any record notebook complete:

```
## Notebook Sign-Off Checklist
[ ] Purpose is clear from the header without opening any code cells
[ ] Every significant decision has a markdown explanation, not just code
[ ] Data source and date of extraction are recorded in the header
[ ] Assumptions and exclusions are explicitly stated
[ ] Output file names make their contents obvious without opening them
[ ] Header accurately reflects what the notebook actually does
[ ] Kernel restarted and all cells run top-to-bottom without error
```

The last point on the header is easy to overlook. A header written at the start of a project often doesn't reflect what the notebook became by the end of it. A notebook titled "exploratory curve QC" that ended up producing the formation completeness dataset used in model training is misleading to anyone who encounters it later. Update the header when the scope changes. It takes thirty seconds and it is the first thing anyone reads.

The cold open test is the difference between a notebook that is technically complete and one that is actually usable. In a field where analysis is regularly revisited, audited, or handed over, that gap matters more than it might seem.

One Habit at a Time

These habits compound, but they don't all need to start at once.

If applying all six on your next project feels like too much overhead, start with the header cell. Write it before you write any code. It takes two

minutes, it forces you to state the purpose and the assumptions before you have committed to any of them, and it is still there when you come back to the notebook in three months wondering what it was for.

The goal is not notebooks that look rigorous. It's notebooks that can be explained, re-run, and handed over without a conversation first. A header cell, honest markdown at each decision point, a clean top-to-bottom run, and sixty seconds asking whether a colleague could follow it cold gets you most of the way there without slowing down the exploration that precedes it.

These habits matter even more as AI starts to enter the analysis workflow. When a model generates an interpretation or a processed output, someone still needs to verify it. That verification depends on having a clear record: what data went in, what decisions shaped it, and what the output actually represents. A notebook without that record doesn't just make your own work harder to return to. It makes AI-assisted work harder to trust. I explored this problem directly in a recent piece on Substack — specifically why verification becomes the binding constraint in technical domains, and what that means for how you build and use AI tools.

→ [The Jagged Frontier: Why AI Amazes and Frustrates in Equal Measure](#)

MT proposal

Mukundan Thanigaivelan

Georgia Doing, PhD

Union College Undergraduate Research - Summer Research Proposal

01/29/2026

Analyzing Different Transfer Functions for Microbiological Transfer Learning

Framework:

One of the most well-studied microbes in the biological literature is *Escherichia coli*, with over 970,000 genomes available in public databases (Ussery, 2024). However, over 99% of living microbes are unknown in science (Jiao et al., 2021). With the few microbes which are known, many including *S. aureus* are less extensively studied. An approach to attempt to understand these other microbes better is transfer learning, where a machine learning model is first pre-trained on a large dataset (such as the one for *E. coli*) and then fine-tuned with one of these microbes' datasets. However, fine-tuning such a model involves a way to map features from the original, larger dataset to features in the smaller dataset. The means of this mapping is through a transfer function, of which there are many different methods.

Our research attempts to evaluate different transfer functions to align *E. coli* features to *S. aureus* features. The different methods that can be used as transfer functions include *k*-mer mapping with tools such as Salmon (Patro et al., 2015), nucleotide-based alignment with BLAST (Healy, 2007), and a higher level protein-based alignment with models such as AlphaFold (Jumper et al., 2021). Different transfer functions can have varying effects on model performance, and we aim to evaluate these functions on which ones assist in training a better model with *E. coli* providing the pre-training dataset and *S. aureus* providing the fine-tuning dataset.

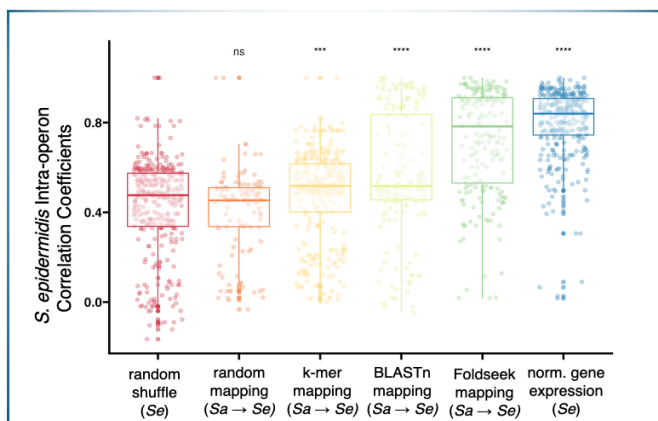


Figure 1 In this side-by-side box plot, *S. aureus* was mapped to *S. epidermidis* through different transfer learning functions. The first two box plots on the left represent negative control groups, either with random mapping or shuffling the genome. The three box-plots afterward represent *k*-mer mapping, nucleotide alignment, and protein-based mapping, from left to right. The last box plot is a positive control group, where *S. epidermidis* was mapped to its own genome. The three functions each had different effects on the model, as seen with the varying correlation coefficients on the vertical axis.

The side-by-side box plot in Figure 1 resulted from a previous analysis on different transfer functions and mapping *S. aureus* to *S. epidermidis*. In this project, we take this research as a precedent for transfer learning and build off of it to map an *E. coli* dataset to a *S. aureus* dataset. The *E. coli* dataset we will use is ten times larger than the pre-training dataset used in

this previous analysis, with over 20,000 samples total. Also, *E. coli* is more distantly related to *S. aureus* in comparison to this past research, which might reveal new insights about transfer learning performance and phylogenetic distance.

Goals:

This project aims to answer the following research **question**: which transfer function produces the best transfer learning performance when fine-tuning a machine learning model pre-trained on *E. coli* with *S. aureus* data?

In the process of answering our research question, we will learn whether different transfer functions behave differently when fine-tuning on the same dataset. We will also examine whether the phylogenetic distance between the original pre-training microbe and the lesser-studied microbe has an impact on the model performance. Model performance will be assessed using both the model's loss function and intra-operon correlations (as in Figure 1), acting as a proxy to capture biological pathways. Overall, we will also look at whether fine-tuning actually benefits the pre-trained model by increasing performance or detracts it by simply adding random noise.

We introduce the main **hypothesis** in regards to our research question: Utilizing *k*-mer mapping as the transfer function will optimize the transfer learning performance of the model, since it works better with more distantly related organisms unlike nucleotide-based BLAST alignment and predictive protein-level mapping, which might only be best for closely related organisms.

Evaluating different transfer functions for model fine-tuning is a specific part of asking a broader question: will fine-tuning actually benefit or disadvantage the model? In one part, introducing a smaller dataset could help with mapping genes and sequence alignment due to similarities in the pre-training and fine-tuning datasets. However, the new dataset could simply introduce noise into the model, weakening rather than strengthening it. This research hopes to address this higher-level question and investigate deeper into what characterizes different functions and their corresponding models.

The anticipated outcomes of this project include building different neural networks on different transfer functions and analyzing their performance, comparing them, and seeing the overall effect of the phylogenetic distance between the pre-training and fine-tuning microbes. This will build off of previous work done with mapping *S. aureus* to *S. epidermidis* with different functions (Tan et al., 2017), and analysis will be used to make conclusions and extract meaningful insights about transfer learning in microbiology.

Impact:

The field of microbiology has vast potential to grow with the study of many unknown and lesser known microorganisms, and one of the main reasons they are less studied is because so little data is available about these microbes. The more that scientists and researchers know about these microbes, the more research that can be done about applications of these microbes to solve or be better prepared for real-world problems.

We aim to see how using transfer learning, where we harness the power of a large dataset combined with a smaller but phylogenetically related dataset, can help the field understand lesser-known microbes better. If we find that this process yields statistically significant results, then future research can focus on utilizing transfer learning and neural networks to study other microbes based on an already well-studied microbe. The world of “microbial dark matter” still remains mysterious with so many unknown microbes, and we hope that our research will take a step towards a better understanding of this interesting but obscure world (Jiao et al., 2021).

Plan:

The main steps for this project can be split into four main stages: pre-training, evaluation, fine-tuning, and then a final thorough evaluation. The first stage will involve training a Denoising Autoencoder (a type of neural network) from scratch using a large *E. coli* dataset (Tan et al., 2017). To do this, we expect to use the Keras Python package (<https://pypi.org/project/keras/>) and build off of other tools already developed for *S. aureus* and *S. epidermidis* (<https://github.com/georgiadoing/seqADAGE>). The second stage, or the first evaluation, will be examining this pre-trained model as a control and assessing its performance, so that comparison can be made to the same model after fine-tuning.

The third part will be fine-tuning the trained model with transfer learning by utilizing different transfer functions discussed earlier. This will be divided into different parts with each using a different transfer function to map *E. coli* features to *S. aureus* features before fine-tuning. Then, the actual fine-tuning will happen, where the model is trained on the smaller dataset. This will produce different neural networks that were trained with different transfer functions for evaluation.

The final part of the project will be evaluation and interpretation. At this stage, we will compare the performance metrics of the pre-trained model only to the transfer learned model to find significant patterns. This evaluation stage will compare both models for *S. aureus* using AureoWiki (<https://aureowiki.med.uni-greifswald.de/>) to obtain pre-determined genes in existing databases as well as EcoCyc (<https://ecocyc.org/>) to assess if the model has learned *E. coli* pathways. From this, we will draw correlations and conclusions about each transfer function, its overall impact on a model's quality, and how it relates to the phylogenetic distance from *S. aureus* to *E. coli*. This process will involve pathway enrichment analysis.

Personal Goals:

I believe that this project supports my long-term career goals by allowing me to work extensively with machine learning and genomics. I am especially interested in using different types of machine learning methods in the scientific field to extract newer insights for real-world problems. Especially in using transfer learning in microbiology, there are many challenges since the feature spaces don't match for *E. coli* and the lesser-studied microbe. The genes are different, the gene names don't match, and genomes vary based on the organism. Microbial transfer learning presents a different real-world problem compared to traditional transfer learning, where

models trained on images of cats can be fine-tuned on dogs since the feature spaces are essentially the same—pixels with x and y coordinates.

Through this research project, I will gain experience with working with neural networks and transfer learning, evaluating different models and how their performance is shaped by input types, and how to conduct independent research and test statistical hypotheses to produce meaningful results. In terms of my professional career, I hope that this project aligns with my long-term goal of applying machine learning techniques in a future career in industry as a machine learning engineer or a data scientist. Overall, I believe that this project will help me develop creative problem-solving skills and experience with machine learning which will help me in pursuing a career in industry. I'm grateful to Professor Georgia Doing for agreeing to advise this project and the Committee on Undergraduate Research for reviewing my research proposal.

References

- Healy, M. D. (2007). Using BLAST for Performing Sequence Alignment. *Current Protocols in Human Genetics*. <https://doi.org/10.1002/0471142905.hg0608s52>
- Jiao, J.-Y., Liu, L., Hua, Z.-S., Fang, B.-Z., Zhou, E.-M., Salam, N., Hedlund, B. P., & Li, W.-J. (2021). Microbial dark matter coming to light: challenges and opportunities. *National Science Review*, 8(3). <https://doi.org/10.1093/nsr/nwaa280>
- Jumper, J., Evans, R., Pritzel, A., & Green, T. (2021). Highly accurate protein structure prediction with AlphaFold. *Nature*, 596, 12. <https://www.nature.com/articles/s41586-021-03819-2>
- Patro, R., Duggal, G., & Kingsford, C. (2015). Salmon: Accurate, Versatile and Ultrafast Quantification from RNA-seq Data using Lightweight-Alignment. <https://doi.org/10.1101/021592>
- Tan, J., Doing, G., Lewis, K. A., Price, C. E., Chen, K. M., Cady, K. C., Perchuk, B., Laub, M. T., Hogan, D. A., & Greene, C. S. (2017). Unsupervised Extraction of Stable Expression Signatures from Public Compendia with an Ensemble of Neural Networks. *Cell Systems*, 5(1), 16. PubMed Central.
- Ussery, D. (2024). E. coli genomes, big data, and messy biology. *Open Access Government January 2025*, 48-49. <https://www.openaccessgovernment.org/article/e-coli-genomes-big-data-and-messy-biology/184909/>

SP proposal

Biological Evaluation of Black Box Models vs. SHAP

Framework and Impact

Artificial Intelligence (AI) has been increasingly applied to solve complex real-world problems, but where we can produce promising results, explanations are less so simple. In most research fields, machine learning (ML) models like neural networks have caused former methods of analyzing data to fade in popularity in favor of ML's superior ability to uncover complex relationships between inputs and outputs in large-scale datasets.

This new shift of method brings forth new complications and in turn, necessary precautions. Whereas the use of ML makes it easier to discern information from datasets, it lacks the ability to provide an explainable result comprehensible to people. Their "black box" nature is particularly an issue for biological research used in health applications where validation and trust of information is required. To combat this, recent advancements in AI research seeks to decipher the model's internal computations by tracing their decision-making processes to understand the key-features driving its predictions.

In particular, post-hoc methods such as certain explainable AI (xAI) models can be used to extract pivotal features from datasets in an attempt to elude a ML model's black box nature. However, model-interpretation and model-explanation algorithms, like xAI, were developed for image processing and other domains, and remain largely untested with biological data. There are also some common concerns of how xAI methods will handle high-dimensional datasets prone to the "curse of dimensionality," where algorithms struggle with high-dimensional data as the data becomes sparse, requiring exponentially more samples to maintain density and leading to a higher risk of overfitting (where a model learns information strictly applicable to the training data, leading to poor performance in real-world applications) (Oldenburg, Jan, et al. 2024).

This can cause unstable interpretability in the extraction of crucial features, resulting in loss of information when extracted and or a misrepresentation of our results. An issue like this would likely be detrimental to research areas using biological data, especially in medical research, where trust and validation of information as well as replicable results are needed. This

is further complicated by the already complex nature of biological data, adding another layer of “dimensionality” to comprehend.

Thus our research intends to test the effectiveness of the xAI method SHapley Additive exPlanations (SHAP) on biological data, in which in our case, is a compendia of approximately 20,000 samples of *Escherichia coli* (*E. coli*) collected from pre-existing publicly available data deposited up until the year 2025. SHAP analysis is one of the more commonly used xAI methods, and provides a view into the inner workings of black-box ML models through calculating the Shapley values (SHAP values) of each feature individually.

SHAP values are numbers representing the contribution of each feature making up the results, giving us an understanding of what key features are contributing to the reconstruction of our input data. Essentially, SHAP values can be understood as the weighted average of a feature’s contribution to all possible outcomes (Ponce-Bobadilla, Ana Victoria, et al. 2024).

Goals

As SHAP is one of the more preferred xAI methods in most other fields of research due to its superiority in offering detailed, consistent, and actionable insights in their outputs, if we can answer what SHAP’s ability to comprehend black box models of biological datasets is, it could help to solve the issue of verification when using black box ML models to process biological data, and possibly allow for better explanation of biological information to be discerned from the dataset’s SHAP values as opposed to what would be extracted from our black box model.

Our aim is to answer if there’s information to be gained from the SHAP values of our black box that would otherwise not be present. In relation to SHAP’s success when applied to datasets of other scientific fields, such as civil engineering and demography, we anticipate SHAP to show similar success when applied to biology.

Plans

To test our hypothesis, we’ll build ADAGE models (a denoising autoencoder for gene expression) to use as our black box model, and we’ll follow a similar process to most other machine learning workflow consisting of three phases: data preprocessing, model training, and

model evaluation. For our data preprocessing phase, our *E. coli* datasets are largely prepared and require minimal work.

To start training our model, there are necessary choices to be made in our model's hyperparameters. Hyperparameters are a configuration variable set before training a model. They define the model's characteristics, dictating the model's structure and the algorithm's learning process. They affect not only the model's performance and resource requirements, but also the biology the model can pick up. Traditionally, hyperparameters are set manually through a process of trial and error, but progress has been made to automate the process, such as AutoML frameworks, to find the hyperparameters that allow for the best performance in the model – in terms of loss minimization – in a more efficient manner (Khan, Muhammad Salman, et al. 2025).

We plan to use KerasTuner to find the hyperparameters of our model. Tuners, like KerasTuner, choose the parameter combinations based on minimizing loss, however, that does not necessarily mean they pick the parameters that allow for the best biology. Thus after the assessment framework of our model has been set up, there is a possibility of continuing our research by testing the optimal hyperparameters that may allow for more biology to be picked up when using SHAP by incorporating biological evaluations into our tuner's framework.

But for now, a few hyperparameters to be considered for our model is the size of the input data, including what portion of the dataset to allocate for training versus the portion to be reserved for testing our model later. Later, when employing SHAP, we'll also have to decide how to go about masking our data for our model and the manner in which we weigh our SHAP values. Once our hyperparameters are established, we can go about training our model. The training phase in its simplest form is a process of multiplying varying weights to the input vector until it has constructed a model with the least amount of error between the input and the reconstructed output. Once our ADAGE models are constructed and frozen, we can use Python's SHAP package to calculate our SHAP value of each sample for each feature.

For our model evaluation phase, we're testing our models performance in biological evaluation and comparing their results. To do so, we're using the models' weight matrices to compare the gene-gene correlations inside to our pre-defined biologically annotated database, EcoCyc, that shows how the genes should be related to each other. The model with more congruence to the annotated database is the model that performs better in extracting biological information.

Personal Goals

My main priority for this project is to practice using machine learning models effectively, particularly when handling biological data. I plan to pursue a career in computational biology, but I lack relevant experience in regards to the practical applications of the field and would like to start building the necessary frameworks. This opportunity would allow me the chance to work towards my goal in a familiar environment with like-minded people.

Works Cited

Khan, Muhammad Salman, et al. "Comparative Analysis of Automated Machine Learning for Hyperparameter Optimization and Explainable Artificial Intelligence Models." *IEEE Access*, vol. 13, 2 May 2025, pp. 84966–84991, <https://doi.org/10.1109/access.2025.3566427>.

Oldenburg, Jan, et al. "XModNN: Explainable Modular Neural Network to Identify Clinical Parameters and Disease Biomarkers in Transcriptomic Datasets." *Biomolecules*, vol. 14, no. 12, 25 Nov. 2024, p. 1501, <https://doi.org/10.3390/biom14121501>.

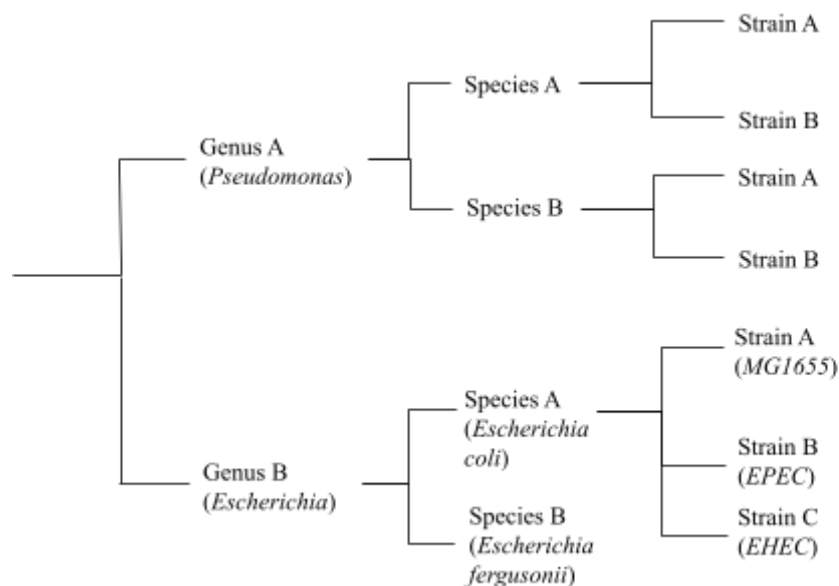
Ponce-Bobadilla, Ana Victoria, et al. "Practical Guide to SHAP Analysis: Explaining Supervised Machine Learning Model Predictions in Drug Development." *Clinical and Translational Science*, vol. 17, no. 11, 28 Oct. 2024, <https://doi.org/10.1111/cts.70056>.

ES proposal

Transfer Learning At The Strain and Genus Levels

Framework

Humans have long been attempting to elucidate the evolutionary relationships between organisms across all levels of biological classification, including species (more specifically different strains) and genera. One way to achieve this is through the use of phylogenetic trees in which branches are used to track the relationships between organisms (that is to say, which organisms are ancestors of which). According to this organizational technique, strains have shorter branches connecting them to their most recent common ancestor, followed by species and thus genera would have the greatest phylogenetic distance between them.



Previously, this tracking was performed by sequence alignment algorithms comparing DNA, RNA or protein sequences. However, with the rise in computer science and biotechnology, researchers have increasingly been able to investigate questions related to phylogenetic inference with various machine learning techniques.

Machine learning is a sector of artificial intelligence centered around pattern recognition. It involves using one dataset to train algorithms to identify patterns and thus make accurate inferences about a different dataset (Bergmann 2025). An example of machine learning is transfer learning which involves two training phases and an assessment phase. The second training phase is used to fine-tune the algorithm in an attempt to optimize performance before assessment. This can be achieved by training the algorithm on two different sets of data which broadens the encoder's ability to identify patterns in the assessment data.

One technique that can utilize transfer learning is a neural network which is a clustering tool that works by applying filters to the training dataset. The input data is provided to layers of neurons (also known as nodes) which turns on an activation function. The network is then trained

on this input data by adjusting the edge weights between adjacent nodes after evaluating the quality of the output (Tan et al. 2017).

An example of a neural network is an autoencoder which can be trained to reproduce the input data. This is a type of unsupervised learning meaning the data that is used to train the model is unlabeled, placing the focus on identifying patterns within it (Mo, Hahn, and Smith 2024). A denoising autoencoder (DAE) is a specific kind of autoencoder in which the input is obscured with added noise.

Despite neural networks proficiency in discerning information from complex datasets, it remains largely untested on model microbes such as *Escherichia coli* (*E.coli*) and *Pseudomonas aeruginosa* (*P. aeruginosa*) despite both having several compendia of their respective gene expression data available publicly. This is partially due to the extent of the performance of machine learning models is unclear, partly due to strain variation. Strain variation refers to changes in genes at the sub-species level which, despite being an important component of understanding the phenotypic variation within a species, inhibits the applicability of gene expression data to an entire species (Lee et al. 2022). Two prevalent disease-causing strains of *E. coli* are *Enteropathogenic E. coli* (EPEC) and *Enterohemorrhagic E. coli* (EHEC) which have increased virulence. These strains are known for their antibiotic-resistance and are also significant causes of mortality, especially in developing countries with less reliable sanitation systems.

Goals

We intend to answer the question: does strain-level transfer learning yield results that are more concordant with biological databases, such as EcoCyc or pseudomonas.com, than species-level and genus-level transfer learning? We will test the accuracy of genus-level transfer learning in DAEs as research at this level of classification has yet to be performed and we already have some unpublished preliminary studies at the species level that suggest its existence (Doing). Of the three levels that we will test, strains are the most genetically similar to each other, followed by species and then genera. Consequently, in our testing of strain-level distances, we hypothesize that transfer learning will produce the most accurate results at the strain-level, then the species level, and the least accurate results at the genus-level. We anticipate that our autoencoder will be able to identify the most statistically significant patterns in the genomes of different strains of the same species. We expect that the DAE-identified patterns will be slightly less accurate when comparing genomes of different, but still closely related, species (in this case, *E. coli* and *P. aeruginosa*) and the least accurate across different genera.

Impact

If our findings suggest that autoencoders can identify statistically-significant patterns across strains, they could serve as evidence that we could bridge the gap between laboratory-studied *E. coli* strains and disease-causing strains. Relations between the EPEC strain and the EHEC strain could be especially important due to their increased virulence. This would

also significantly improve our understanding of this bacteria and allow us to develop a more efficient public health response.

Additionally, providing some evidence that transfer across genera is possible could further advance the field of computational biology. It would create a basis for further investigations to test the extent to which machine learning can be transferred and could solidify existing studies conducted across genera. One area that would be especially impacted includes studies conducted on laboratory animals, such as mice, in which the researchers have attempted to generalize the results to humans. It would also provide a basis for extending existing findings to understudied organisms. This would allow us to gain a deeper understanding of a variety of organisms that are difficult to study in a laboratory setting for a variety of reasons such as limited resources.

This research will also advance the field of computational biology towards being able to ask other questions regarding phylogenetic inference. We currently only have data sets for certain organisms, but with further investigation, this research could be extended to a larger sample and greater variety of organisms. This could allow us to determine the maximum possible phylogenetic branch length between microbes at which our denoising autoencoder can recognize biologically-driven gene expression patterns.

Plan

For each condition that we test, there will be two sections of the experiment: the first section involves the experimental portions (a data setup phase and a transfer phase) and the second section consists of analyzing the output of the autoencoder. We have a compendium of approximately twenty thousand samples of *E. coli* with strain-level labels that already existed and consists of all publicly available data as of 2025. We also have a compendium of approximately two thousand samples of *P. aeruginosa* that already existed and consists of all publicly available data as of 2023 (Doing, Georgia et al. 2023). Samples in both compendia are from a variety of conditions, including, but not limited to, different studies, temperatures, media and strains. In the data setup phase, we will use the *E.coli* and *P. aeruginosa* compendia to develop criteria to determine which samples will be used for which step of the transfer phase.

The transfer phase will include three parts: pre-training, fine tuning and assessment. A different set of data samples will be used in each phase depending on which experimental condition we are testing at that time. Some conditions include comparing strains of *E. coli* by pre-training on the MG1655 strain and fine tuning it on other strains such as *Enterotoxigenic E. coli* and DH10B, comparing the EHEC and EPEC strains, and pre-training on *E. coli* samples and fine tuning using *P. aeruginosa* samples. Our model will be constructed using the Keras Python library.

Given that a DAE is a clustering technique, our analysis will consist of looking for enrichment between the output and known pathways on pseudomonas.com and KEGG pathways (well-known gene pathway databases). This mapping between each gene and pathway will be used to evaluate the biological relevance of the gene signatures (Tan, Jie, et al. 2017). We will

also compare outputs across different pairs of pre-training and fine-tuning datasets. For example, we will compare the results from our *E. coli* strain condition to those of our *E. coli* and *P. aeruginosa* test to determine the strain-level workflow and the genus-level workflow of our DAE.

Personal Goals

As a neuroscience major, one of the areas that I am deeply interested in is the overlap between biology and computer science. Thus this research is the perfect opportunity for me to expand my own knowledge while simultaneously contributing to the discipline and society as a whole. I also hope to one day work in the field of bioinformatics and this opportunity would serve as an incredible introduction to the technical aspects of the field which are often more difficult to learn in a lecture-based classroom setting.

Works Cited

- Bergmann, Dave. "What Is Machine Learning?" *IBM*, 17 Nov. 2025, www.ibm.com/think/topics/machine-learning.
- Doing, Georgia et al. "Computationally Efficient Assembly of *Pseudomonas aeruginosa* Gene Expression Compendia." *mSystems* vol. 8,1 (2023): e0034122. doi:10.1128/msystems.00341-22
- Doing, Georgia. "Georgiadoing/seqADAGE." *GitHub*, github.com/georgiadoing/seqADAGE. Accessed 4 Feb. 2026.
- Lee, Alexandra J., et al. *Using Genome-Wide Expression Compendia to Study Microorganisms*. 20 Vol. Elsevier BV, 2022. Web.
- Mo, Yu K., Matthew W. Hahn, and Megan L. Smith. "Applications of Machine Learning in Phylogenetics." *Molecular phylogenetics and evolution* 196 (2024): 108066. Web.
- Tan, Jie, et al. *Unsupervised Extraction of Stable Expression Signatures from Public Compendia with an Ensemble of Neural Networks*. 5 Vol. Elsevier BV, 2017. Web.